

Harbormaster



AI Systems Architecture Review

Intelligent file-arrival control for financial reconciliation — a three-tier mapping ladder, quarantine and four-eyes review, hash-chained audit

Ship readiness complete · 12 September 2026 · Roshan Rana, AI Systems Architect

Executive summary

What it is

- The control layer between a bank's FTP landing directory / JSON queue and its reconciliation engine: six event-driven services (Go + Python) that decide which client a file belongs to, which value date it describes, and how its columns map onto one canonical schema.
- It declines when it cannot decide, quarantines for a human, and records why.

What it proves

- A three-tier mapping ladder — alias dictionary, then fuzzy match, then a model — whose model dependence falls over time as confirmed mappings are cached and promoted into configuration.
- 97.7% field-mapping accuracy and 80.8% resolved without a model, measured against a 214-column hand-labelled corpus by a bench harness that reproduces the same numbers on every run.

Why it is enterprise-ready

- Hash-chained, tamper-evident audit (an edited or deleted record is detected and the chain fails closed); quarantine with four-eyes review; masking of account numbers by default.
- 20 architecture decisions, 50 task packs, twelve named adversarial fixtures, 347 tests passing offline in this run, CI running the same gate.

Ask of the audience: agree the pilot scope (host-validated Docker run of make up / make test-e2e / HM_LIVE=1 make test-live, load test against the NFR-2 latency target) and whether Tier 2 embeddings (fastembed) ship enabled or degraded.

The problem and the users

The bank's problem

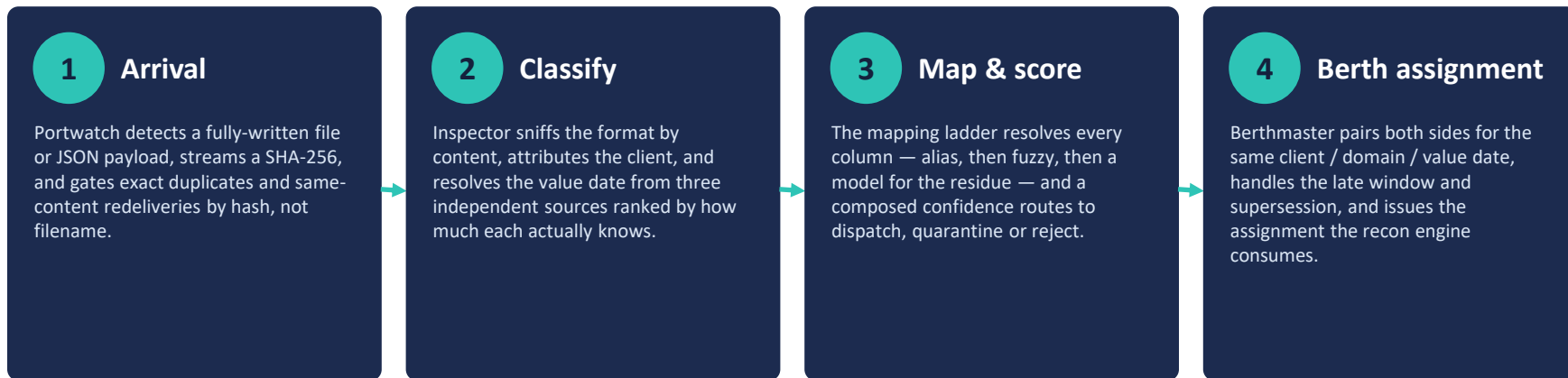
Client, custodian and exchange reports land all day into one FTP directory as CSV, Excel, XML, FIXML and free-text email, plus a parallel JSON queue. Every venue names its columns differently, filenames are unreliable, and a file produced late at the client carries yesterday's business under today's date.

The engine's constraint

The downstream reconciliation engine is deliberately simple: it matches records for an expected file on each side. It has no opinion about which client, which value date, or which of four price columns is the trade price. It has to be told — correctly, or not at all.

Actor	Needs	Frequency
Reconciliation engine (downstream)	One canonical Parquet schema per file, with client, value date and side already resolved	Continuous, all day
Four-eyes reviewer	A specific reason for every quarantine, and the ability to correct, release or reject	On quarantine (0% of well-formed fixtures)
Client onboarding / ops team	Per-client YAML config; a promoted mapping resolves at Tier 1 forever after	As new clients or venues arrive
Compliance / auditor	A tamper-evident decision log that reconstructs any dispatch after the fact	On review, or on demand

Solution at a glance



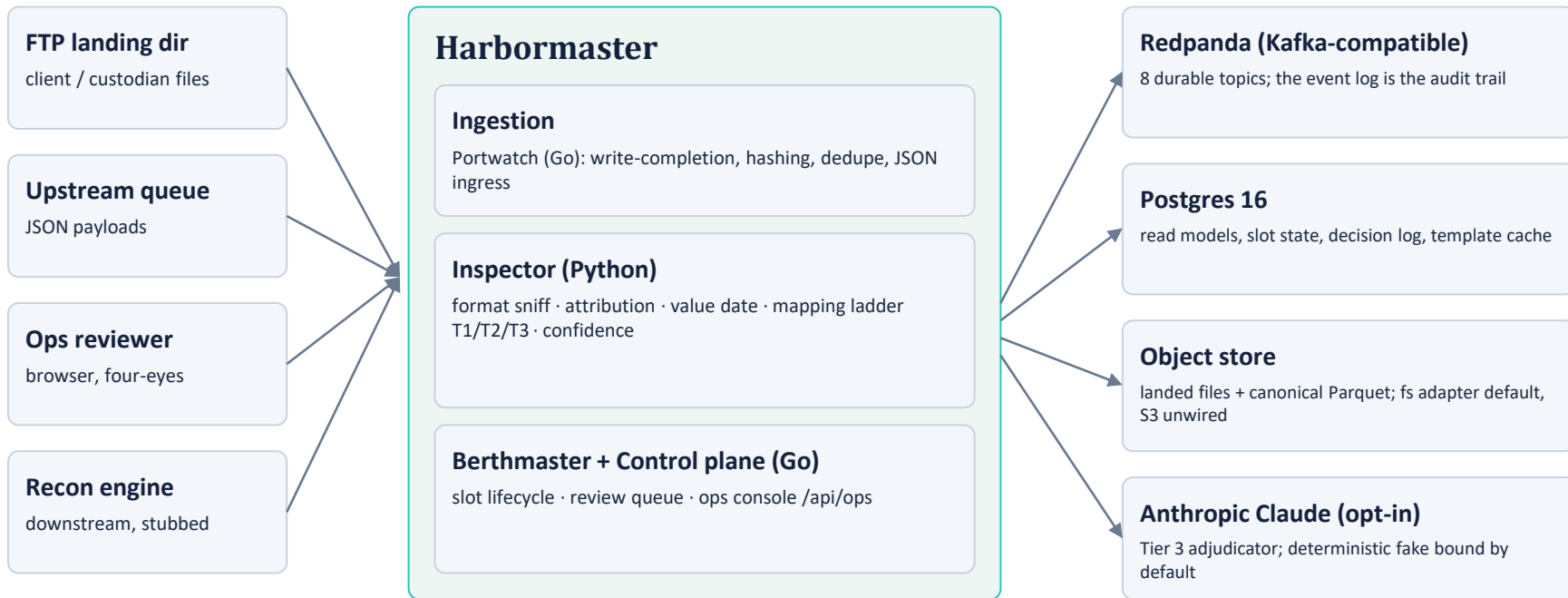
Six services, one log

Portwatch, Inspector, Berthmaster and Control plane over Kafka-compatible topics (Redpanda), a review queue with four-eyes decisions, and an operations console at /api/ops. Everything is replayable from offset zero.

Deterministic by default

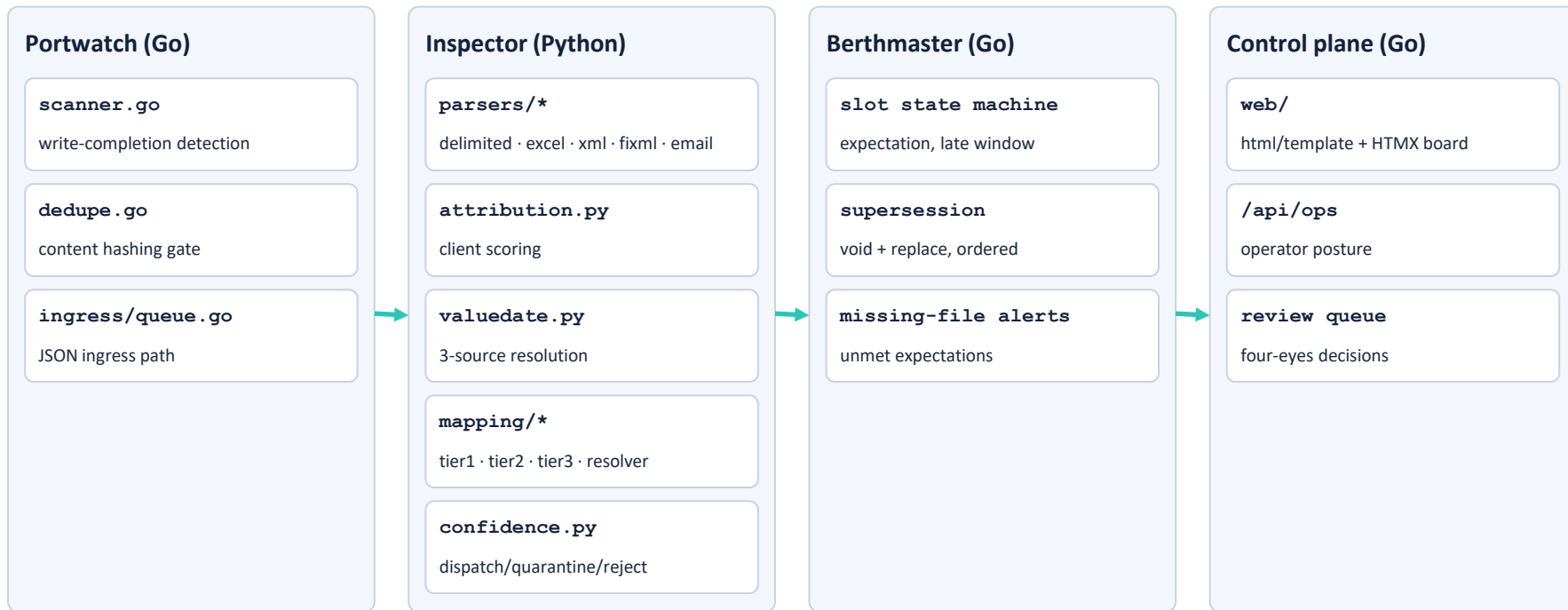
The model tier is bound to a deterministic stand-in unless HM_ADJUDICATOR=claude and ANTHROPIC_API_KEY are set. make check and make bench run fully offline: no broker, no database, no network, no key.

System context



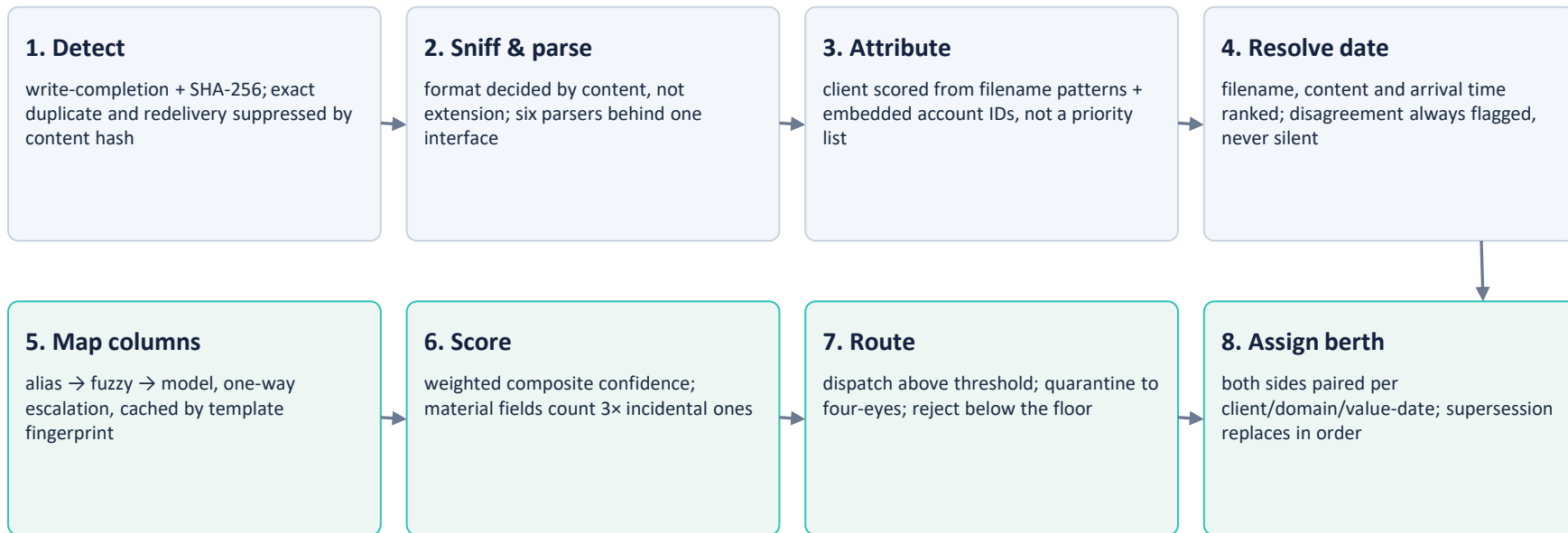
Docker (Redpanda, Postgres, all six services) is host-validated: not run in this build environment (ADR-018). The gate and the bench harness never touch a network.

Component architecture



Message contracts are defined once in proto/ and generated for both languages with buf; protojson on the wire keeps topics human-readable for triage.

The arrival-to-berth-assignment pipeline



Every stage records why it decided what it did. Audit is a property of the architecture: each decision appends to a hash-chained log, and read models are projections rebuildable from offset zero.

The mapping ladder: falling model dependence

Three escalating tiers, one-way

T1	alias dictionary (us)	160/214	74.8%
T1	structural price rule	4/214	1.9%
T2	fuzzy match (ms)	8/214	3.7%
	client override (promoted)	1/214	0.5%
T3	model, given the residue	5/214	2.3%
	unresolved, declined	36/214	16.8%

- Each tier sees only what the previous one declined; a column T1 resolved is never re-litigated by a model
- Results are cached by template fingerprint — the second file with a known layout costs nothing
- A reviewer can promote a confirmed T3 decision into the client's T1 dictionary; it resolves at T1 forever after

Structure over vocabulary

settlement/mark/prior-settlement repeat per instrument-day; trade price varies per row — resolved by that shape, not the header text

Provenance on every field

each dispatched mapping carries a tier and non-empty evidence string; a mapping with no provenance is a defect

Decline over guess

Measured, not hoped

Second file with a known layout costs zero adjudications — asserted in `test_correctness_targets_across_the_corpus`, not claimed in prose.

Data architecture

8 Kafka-compatible topics

hm.arrivals.raw (Portwatch→Inspector)
hm.arrivals.classified (30d)
hm.arrivals.quarantined (30d)
hm.review.decisions (30d)
hm.berth.assigned (90d)
hm.berth.superseded (90d)
hm.alerts.missing (30d)
hm.audit.decisions (compacted, infinite)

Partitioned by client:domain:value_date — ordering within a slot is structural, not a runtime hope.

Postgres 16

Read models (arrival board, slot state), expectation state, hash-chained decision log, template mapping cache, review decisions. JSONB for variable-shape payloads.

Canonical output: Parquet, not CSV

Typed and columnar; a null settlement price survives distinctly from a zero one, which the downstream engine must not confuse (ADR-006).

Object store

Landed raw files + canonical Parquet output, behind an interface. Filesystem adapter is the default; an S3 adapter is implemented and unit-tested but unwired in the demo (ADR-015).

Masking

Account numbers masked to last-4 in logs and UI by default; account and client names stay visible. Applied in a serialisation hook, not per call site (ADR-011).

Contracts are defined once in proto/ and generated for Go and Python by buf; a cross-language round-trip test is part of the offline gate.

The model tier: opt-in, behind an interface

Off by default

HM_ADJUDICATOR defaults to fake — a deterministic stand-in bound behind the same MappingAdjudicator interface Claude implements. A misconfigured deployment runs without a model rather than failing to start.

Opt-in, explicit

HM_ADJUDICATOR=claude ANTHROPIC_API_KEY=sk-... make up enables the real tier; HM_LIVE=1 make test-live is the opt-in test against the live API. Every prior phase runs against the deterministic fake.

Bounded cost

Escalation is one-way; results are cached by template fingerprint. The second file with a given layout is resolved from cache and costs nothing — watch HM_ADJUDICATOR=claude logs for call and token counts.

What is and is not measured

Offline: which columns fall through to T3, and whether the stand-in and cache behave correctly (41 columns reached T3 here; 18 adjudication calls). Live-model mapping accuracy itself is pending — it needs the real key and is marked so in metrics/headline.json.

Design choices and why

Decision	Alternatives considered	Why this one
Three-tier mapping, not LLM-first (ADR-004)	A model call per column	1,000 files/day × 10–40 columns makes per-column LLM calls slow, non-deterministic and unbounded in cost
Polyglot split: Go at the edges, Python in the middle (ADR-002)	Single-language stack	Portwatch/Berthmaster are I/O and state machines; Inspector is data shape and inference — each gets the right tool
Protobuf as the single source of truth (ADR-003)	Hand-written structs per language	Two languages on one topic will drift; proto-as-source turns that into a build failure, not a production incident
Poll the landing directory, not inotify (ADR-016)	fsnotify / inotify watcher	Landing dirs are NFS/SMB mounts; inotify silently misses remote-host writes exactly where it matters most
Server-rendered HTMX, not a React SPA (ADR-012)	React/TS/Vite front end	No Node, no bundler, no second dependency tree, for a live board plus a review form
fastembed (ONNX) over sentence-transformers (ADR-007)	sentence-transformers + PyTorch	~90 MB CPU-only image instead of ~2 GB; degrades to fuzzy-only if unavailable, and says so
Topics provisioned by infrastructure, not the app (ADR-020)	Broker auto-create topics	A typo in a topic constant now fails loudly instead of silently stranding messages in a new topic

Major features

The mapping ladder

Alias dictionary → fuzzy/embedding match → model adjudication for the residue, with provenance on every field and promotion into client config

Value-date resolution

Three independent sources ranked by what they know, resolved against per-client business calendars; disagreements always flagged

Quarantine & four-eyes review

Composite confidence routes to dispatch, quarantine or reject; no quarantined arrival ships without a recorded human decision

Hash-chained audit

Every classification and dispatch decision appends to a tamper-evident chain; a verifier walks it and reports the first break

Slot lifecycle & supersession

Expectation state machine per client/domain/value-date, a $T \pm 2$ late window, and ordered void-then-replace on correction

Operations console

Embedded Go templates + HTMX over Postgres read models, projected from the topics; /api/ops for whoever is on call

The offline gate, green

```
$ cd go && GOWORK=off go vet ./...  
(clean - no output)  
  
$ cd python && uv run ruff check . && uv run ruff format --check .  
All checks passed!  
48 files already formatted  
  
$ uv run python -m mypy src  
Success: no issues found in 33 source files  
  
$ cd go && GOWORK=off go test ./... -count=1  
ok   harbormaster.dev/hm/internal/arrival    0.241s  
ok   harbormaster.dev/hm/internal/audit      1.272s  
ok   harbormaster.dev/hm/internal/bus         1.029s  
ok   harbormaster.dev/hm/internal/contract    0.856s  
ok   harbormaster.dev/hm/internal/ingress     0.194s  
ok   harbormaster.dev/hm/internal/logging     0.488s  
ok   harbormaster.dev/hm/internal/masking     0.529s  
ok   harbormaster.dev/hm/internal/registry    0.883s  
ok   harbormaster.dev/hm/internal/slot        1.111s  
ok   harbormaster.dev/hm/internal/store       1.163s  
ok   harbormaster.dev/hm/internal/store/objectstore 0.508s  
ok   harbormaster.dev/hm/internal/web         0.174s  
73 passed, 47 skipped (require HM_TEST_POSTGRES_DSN - live Postgres), 0 failed  
  
$ cd python && uv run python -m pytest -q  
257 passed, 5 skipped in 4.8s (262 collected - tests/)  
  
$ uv run python -m pytest -q ../tests/scenarios  
17 passed in 1.7s - the twelve adversarial fixtures, each with a named test  
  
Total observed today: 347 passed / 399 collected, 0 failed, 52 skipped (need a live Postgres)  
proto-check: SKIPPED - protoc/buf not installed on this host (pre-existing gap, not fixed here)  
golangci-lint: not installed on this host (go vet substituted above)
```

go vet, ruff, mypy --strict, go test, and both pytest suites run in this task: 347 passed, 0 failed, 52 skipped (need a live Postgres). proto-check needs protoc/buf, absent on this host (pre-existing, not fixed here).

The bench reproduces every published number

```
$ cd python && uv run python ../bench/onboard.py --out ../metrics/headline.json
mapping_accuracy          97.7%  209/214 labelled columns across 19 files, against bench/golden
no_model_share            80.8%  alias, structural, fuzzy and promoted overrides: 173/214 columns
tier3_escalations         41    18 adjudication calls; stand-in answered 5, cache replayed 0; not a live-model accuracy
quarantine_precision      100.0% 2 held correctly, 0 held needlessly, of 21 classifications
quarantine_recall         66.7%  2 of 3 files that should be held were held; 1 slipped through
tamper_check              detected, fail-closed edited outcome caught at record 2; deleted record caught at record 2
  Tier 1: alias dictionary          160/214 (74.8%)
  Tier 1: structural price rule     4/214 (1.9%)
  Tier 2: fuzzy match               8/214 (3.7%)
  Client override (promoted)       1/214 (0.5%)
  Tier 3: model (deterministic fake) 5/214 (2.3%)
  Unresolved (declined)           36/214 (16.8%)
wrote C:\Code-Central\harbormaster\metrics\headline.json

# offline, seeded (20260831), sha256-pinned source files — no broker, no DB, no API key
```

python bench/onboard.py, run in this task, wrote metrics/headline.json byte-for-byte identical to the committed file — the README's 97.7% / 80.8% / 100.0% / 66.7% figures are reproducible, not asserted.

The mapping ladder deciding, and quarantine holding

```

ARRIVAL DETAIL -- four_price_columns
=====
file:          07_four_prices\ICE_TRD_20260828.csv
client:        CLNT004 (confidence 0.95)
value date:    2026-08-28 (method: CONTENT_EXPLICIT)
disposition:   DISPATCHED (overall confidence 0.97)

source column      canonical field    tier    evidence
-----
ClearingAccount     account_id      ALIAS   alias:clearingaccount
ContractCode        instrument_id    ALIAS   alias:contractcode
DealID              trade_id        ALIAS   alias:dealid
TradeDt             trade_date      LOCAL   fuzzy match to alias vocabulary at 0.88
SettlementDate      (unresolved)    UNRESOLVED no alias match; tier2: best fuzzy 0.57; tier3 declined
Contracts           quantity        ALIAS   alias:contracts
Price               trade_price     LOCAL   varies per row within instrument groups (ratio 1.00)
Settle Px           settlement_price ALIAS   header names a settlement or mark price
Mark Price          (unresolved)    UNRESOLVED also names a settlement price; 'Settle Px' is the stronger match
Prior Settle Px     (unresolved)    UNRESOLVED also names a settlement price; 'Settle Px' is the stronger match
NetMoney            net_amount      ALIAS   alias:netmoney
Currency            currency        ALIAS   alias:currency
BuySell             side            ALIAS   alias:buysell

Settlement / mark / prior-settlement resolve by structure (the value repeats
across rows for the instrument-day); trade price varies per row.

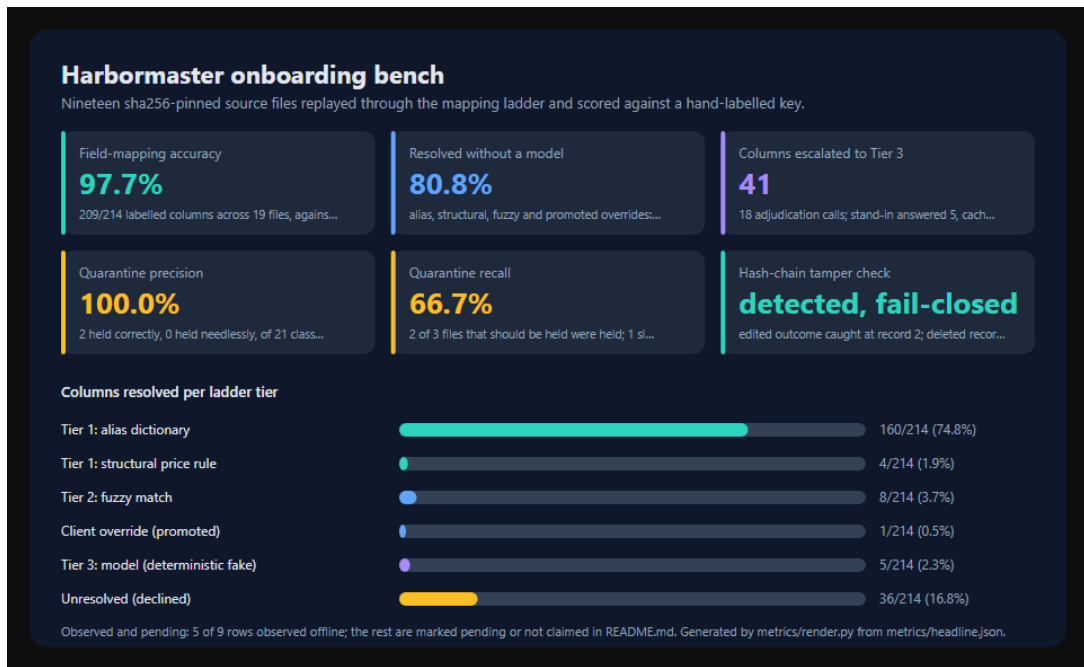
=====
QUARANTINE QUEUE -- unattributable_file
=====
file:          10_unattributable\export_final.csv
disposition:   QUARANTINED
client conf:   0.00 mapping coverage: 1.00
overall:       0.49 (quarantine band: 0.60-0.85)
reasons:
- client could not be determined: no filename pattern matched and no known account identifiers were found
- value date rests on weak evidence (ARRIVAL_MINUS_CALENDAR, confidence 0.55)

# Four-eyes review queue: held for a human decision, never auto-dispatched (FR-32)

```

A small script run against the real Inspector for this task: four price columns disambiguated by structure, and the one deliberately unattributable fixture held in the four-eyes queue with its exact reasons.

The generated metrics card



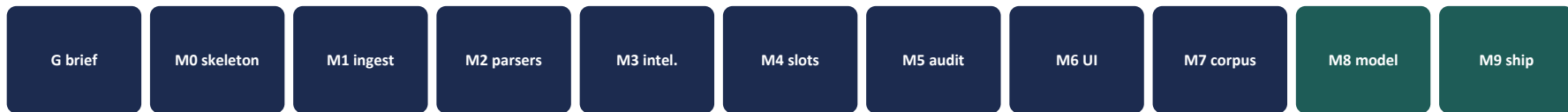
metrics/render.py renders this card from metrics/headline.json into docs/assets/metrics.svg and the README results block, so the deck, the README and the bench output cannot drift apart.

Security and compliance controls

Boundary	Threat	Control in the code	Evidence
B1 landing directory	malformed/oversized files, format confusion	content-based sniffing, not extension; streaming hash; per-format parsers behind one interface	arrival + parser tests
B2 model tier (Claude)	key leakage; unbounded cost; wrong mapping accepted	key from env only; one-way escalation; cache by template; structured-output validated against canonical schema	T3/resolver tests, ADR-004
B3 operations console	no authentication in the prototype	explicitly out of scope by requirements §7; designed to sit behind the institution's SSO	ship report §5, item 7
B4 audit log	tampering, deletion, silent edit	SHA-256 hash chain per arrival; a verifier walks every chain and reports the first break; fails closed	internal/audit tests, bench tamper check
B5 logs and UI	account numbers or client identifiers leaking	numbers masked to last-4 by default in a serialisation hook; names stay visible for operability (ADR-011)	masking + parity tests, Go & Python

Residual risks, written down rather than hidden: no console authentication (by design, pre-SSO), no leader election on the overdue sweep, throughput designed for but not load-tested.

How it was built: a gated, evidence-producing lifecycle



50

task packs, each with scope, acceptance criteria and a validation command

20

architecture decisions, each with context, consequences and cost

12

named adversarial fixtures, each with its own scenario test

347

tests passing offline in this run: 0 failed, 52 skipped pending a live Postgres

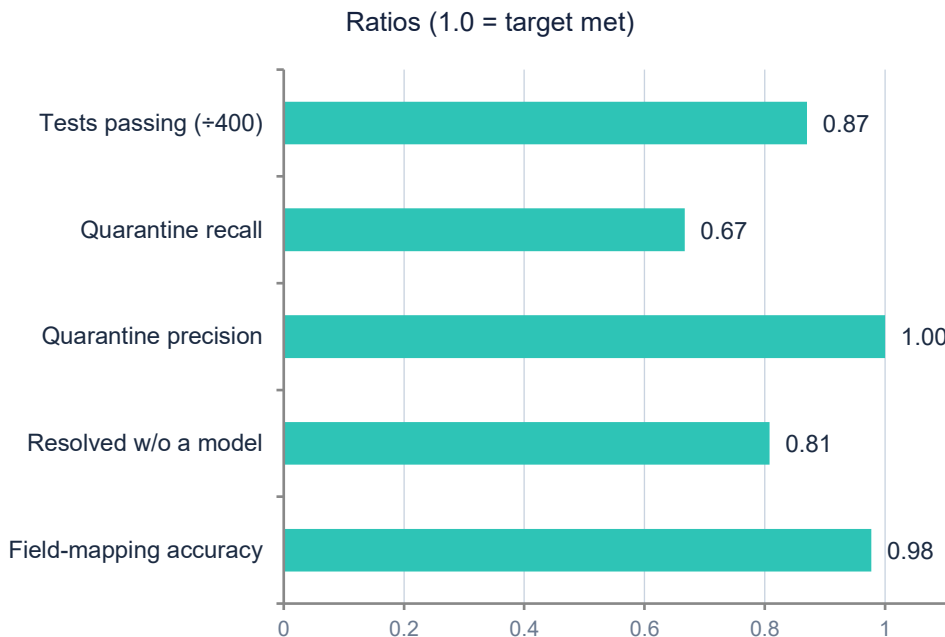
Split by what the build host can run

Every task is marked validated-here or validate-on-host (ADR-018). Docker (Compose, Redpanda, the six-service stack) is absent from the build environment, so end-to-end and live-model gates are authored to spec and run on a host with Docker.

One command validates the rest

gofmt · go vet · golanci-lint · ruff · mypy --strict · buf lint + codegen staleness · a cross-language contract test · every unit, property and scenario test. CI runs the same make check target verbatim.

Quality metrics from the bench and the gate



KPI	Value	Target
Field-mapping accuracy	97.7% (209/214 columns, 19 files)	n/a, headline
Resolved without a model	80.8% (173/214)	n/a, headline
Columns escalated to Tier 3	41 (18 adjudication calls)	≤2% once warm (NFR-8)
Quarantine precision / recall	100.0% / 66.7%	n/a; recall is an honest miss
Hash-chain tamper check	detected, fail-closed	must fail closed
Tests today (this run)	347 passed / 399 collected, 0 failed	offline, no network
make check (minus proto-check)	green in this run	green offline

Quarantine recall (66.7%) is shown as measured, not smoothed: one of three files that should have been held slipped through in this corpus — see Quirks. Tier-3 escalation count is against a deterministic stand-in, not a live model.

Quirks and known limitations (stated, not hidden)

Quarantine recall is 66.7%, not 100%

1 of 3 files that should have been held was not, in the fixed reference corpus. Precision is 100% (nothing held needlessly) but recall is the honest miss; mitigation is a lower quarantine threshold or an added evidence source, not yet done.

Tier 2 embeddings are optional and untested against a real model here

fastembed is not installed in this environment, so the suite exercises the fuzzy-only degraded path; the degradation is recorded in the evidence string, never silent.

Live model accuracy is not measured offline

The Tier 3 numbers above are against the deterministic stand-in. HM_LIVE=1 make test-live against the real Claude API is written but not run in this environment.

Docker-dependent gates are host-validated, not run here

make up, make test-e2e, make demo and make verify-audit need Redpanda/Postgres/the six-service stack; ADR-018 marks every such task validate-on-host rather than claiming it done.

No authentication on the operations console

Out of scope by requirements §7; intended to sit behind the institution's existing SSO for a pilot.

The overdue sweep is not leader-elected

Running more than one Berthmaster instance would run the sweep more than once per interval. Slot state itself is safe across instances.

Throughput and p95 latency (NFR-2) are designed for, not measured

The 1,000 arrivals/day envelope has no load test behind it in this environment.

golangci-lint and protoc/buf are not installed on this host

go vet substituted for lint; proto-check could not run. Pre-existing gaps in this environment, not fixed as part of this task.

Cost, performance and operability

μs / ms / s+\$

the three tiers' cost, in order — most columns resolve for free

0 keys

needed for make check or make bench;
the model tier is opt-in only

~4.8s

python/tests (257 passed) in this run;
go test ./... a few seconds more

0 → next time

template-fingerprint cache: the second file with a known layout costs no model call

Operability

- Runbook: first checkout, running the stack, seeing it work, verifying the audit trail, configuration reference, what to do when something is wrong
- make up targets healthy in under a minute (host-validated)
- make demo narrates the twelve adversarial fixtures through the live board
- Rollback: read models are projections, rebuilt by replaying topics from offset zero

Observability and audit

- Structured JSON logs with correlation IDs (slog / structlog); no metrics stack by design (ADR-014)
- Hash-chained audit.decisions topic, compacted and retained indefinitely
- make verify-audit walks every chain and names the first break
- Client YAML is schema-validated strictly at load: an unknown key fails loudly, never at 3 a.m.

Roadmap to production validation

Now

Host validation

- go mod tidy; make up && make test-e2e on a machine with Docker
- HM_LIVE=1 make test-live against the real Claude API
- Confirm the 60s p95 / 180s p99 latency targets (NFR-2)

Near term

Close the recall gap

- Investigate the missed quarantine case; consider a lower threshold or an added evidence source
- Install fastembed and re-measure Tier 2 with real embeddings, not fuzzy-only

Before pilot

Operational hardening

- Leader-elect the Berthmaster overdue sweep for multi-instance runs
- SSO in front of the operations console
- Load test the 1,000 arrivals/day envelope and the burst target

Scale

Beyond the prototype

- Wire the already-implemented S3 object-store adapter
- Extend the client dictionary from pilot-onboarding promotions
- Revisit Redpanda vs. managed Kafka for production topology

Every item above is additive: the contracts, the ladder, the confidence scoring and the audit chain do not change.

Appendix A — Decision log (abridged, 20 total)

ID	Decision	Consequence
ADR-001	Event-driven log backbone (Kafka-compatible) over batch or sync calls	Replay is an offset operation; the log is the audit trail
ADR-002	Polyglot split: Go at the edges, Python in the middle	Each language fits its workload; a hard requirement for shared contracts
ADR-003	Protobuf single source of truth, buf-generated, protojson on the wire	Contract drift becomes a build failure, not a production incident
ADR-004	Three-tier field mapping, not LLM-first	Model dependence decreases over time; NFR-8 ($\leq 2\%$ once warm) achievable
ADR-006	Parquet for canonical output, not CSV	Nulls and types survive; a missing price is never confused with zero
ADR-011	Masking: numbers masked to last-4, names visible	Boards and screenshots stay safe without hiding who a client is
ADR-016	Poll the landing directory, not inotify	Correct on the NFS/SMB mounts this actually runs on
ADR-018	Split validation between build and host environments	No task marked done on unexecuted code; the gap is visible, not implied
ADR-020	Topics provisioned by infrastructure, not the application	A topic typo fails loudly instead of silently stranding messages

Appendix B — Build statistics and defects found

Measure	Count
Go source	4,991 lines across 12 packages and 3 binaries
Python source	6,295 lines
Go tests	73 passed, 47 skipped without a live Postgres, in this run
Python tests	257 passed / 262 collected (tests/), 17/17 (scenarios), in this run
Documentation	2,877 lines across requirements, HLD, LLD, ADRs, runbook, ship report
Task packs / ADRs	50 task packs; 20 architecture decisions

Defects found and fixed during construction (from the ship report)

- Audit hashes recomputed differently on read: Postgres stores microseconds, Go carries nanoseconds — every chain would have failed verification
- inputs/outcome stored as JSONB normalised key order, so hashed bytes did not match stored bytes — changed to TEXT
- A description column full of commas beat the real pipe delimiter in the delimiter scorer
- bv or "" in the recon stub made a genuine zero indistinguishable from null (Decimal("0") is falsy)
- Masking revealed short account numbers in full: "last four" of a four-character value is all of it

Appendix C — Repository map and how to run

```
go/          portwatch, berthmaster, controlplane, auditcheck,
             internal/{arrival,audit,bus,contract,ingress,
             logging,masking,registry,slot,store,web}
python/      inspector (parsers, mapping, valuatedate,
             attribution, confidence), corpus, reconstub
proto/       harbormaster.proto – single source of truth
config/      clients/*.yaml, calendars/*.yaml
bench/       onboard.py, golden/ – the fixed labelled corpus
metrics/     headline.json, card.json, render.py (drift-checked)
tests/scenarios/ the twelve adversarial fixtures, one test each
docs/design/  requirements, HLD, LLD, execution plan, 20 ADRs
docs/tasks/   50 task packs
docs/pitch/   this deck
```

Run it

```
cd go && go mod tidy
cd .. && make check
make bench && make card
make test-scenarios

# with Docker (host-validated)
make up
make demo
make verify-audit

# the live model tier
export HM_ADJUDICATOR=claude
export ANTHROPIC_API_KEY=sk-...
HM_LIVE=1 make test-live
```

Questions

Harbormaster · roshanrana/harbormaster · design docs, decision log and runbook in the repository